

Genetic Programming

A Simple Introduction

An easy, example-based guide for undergraduate students

Dr. Kanchan Rajwar

Department of Mathematics

Indian Institute of Information Technology, Bhopal, India

Email: kanchan.rajwar@iiitbhopal.ac.in

Optimization Techniques • CSE(DS)-5003 • Module 3

To the Student

This small book explains Genetic Programming (GP) in simple words. It is written for students who are meeting the topic for the first time. It is a companion to the Genetic Algorithm (GA) notes of the same module, so it is helpful to read the GA notes first. Every idea is built slowly — first we see why the idea is needed, then we learn the idea, then we work through an example. You do not need advanced mathematics.

How to use these notes:

1. Attend the class lecture for each topic.
2. Read the matching section of these notes carefully, working through each example yourself.
3. After a chapter, attempt the related questions in the companion question bank (GP Module 3 — Question Bank) for self-assessment.
4. Check your answers only after attempting the questions on your own.

Regular self-assessment is one of the best ways to prepare for exams. Do not skip it.

Good luck, and enjoy the topic.

Dr. Kanchan Rajwar

Contents

Preface	4
1 Introduction	5
1.1 From Numbers to Programs	5
1.2 Learning from Nature	5
1.3 What is Genetic Programming?	5
2 Basic Terms	6
2.1 Fitness	6
3 How GP Works	8
3.1 Tree Representation	8
3.2 Terminal and Function Sets	9
3.3 Creating the Initial Population	9
3.4 The Fitness Function	9
3.5 Selection	9
3.6 Crossover (Subtree Swap)	10
3.7 Mutation	10
3.8 A Complete GP Run	10
3.9 Bloat and Its Control	11
3.10 Parameters and Practical Guidelines	11
4 Advantages and Limitations	12
4.1 Advantages	12
4.2 Limitations	12
5 Conclusion	13
Python Implementation	14

Preface

This book is a module-level introduction to Genetic Programming, prepared for students of Optimization Techniques (CSE(DS)-5003) at the Indian Institute of Information Technology, Bhopal. It forms part of Module 3 of the course, alongside the notes on Genetic Algorithms.

The notes are written to be self-contained. A student who reads carefully, works through every example, and attempts the companion question bank should be well prepared for examination questions on this topic.

The running example — fitting the data of the target function $t(x) = x^2 + 1$ using trees built from the terminal set $\{x, 1\}$ and the function set $\{+, -, \times\}$, with a population of four trees — appears throughout the book. This consistency is deliberate. It lets the reader see exactly how each operator acts on the same data, so the method is understood as a whole rather than as a set of isolated steps.

Dr. Kanchan Rajwar

Department of Mathematics

Indian Institute of Information Technology, Bhopal, India

kanchan.raajwar@iiitbhopal.ac.in

August 2026

1. Introduction

In the previous module we studied the Genetic Algorithm (GA). A GA evolves a *value* — for example, the number x that maximises $f(x)$. The candidate solutions are fixed-length strings. But sometimes the answer we want is not a value. It is a whole *formula* or a small *program*. Genetic Programming (GP) is the branch of evolutionary computation that evolves such programs.

1.1 From Numbers to Programs

Consider a common task called *symbolic regression*. We are given some data points, and we must find a formula that fits them. We do not know the shape of the formula in advance. It could be $x + 1$, or x^2 , or something larger.

A GA cannot solve this directly. Its chromosome is a fixed-length string, so it cannot grow a formula whose size is unknown. We need a representation that can grow and shrink freely. A *tree* is exactly such a representation, and GP evolves trees.

1.2 Learning from Nature

Like the GA, GP borrows its idea from natural evolution. It keeps a population of candidate programs. It scores each program with a fitness value. Fit programs are selected, combined, and slightly changed to produce the next generation. Over many generations, the programs improve. The only new idea is the representation: the candidate solution is now a tree, not a string.

1.3 What is Genetic Programming?

A **Genetic Algorithm** evolves fixed-length strings that encode a solution. A **Genetic Programming** system evolves variable-size trees that encode a program or formula. Both use the same three operators — selection, crossover, and mutation — and the same main loop.

GP keeps the strengths of the GA. It needs no gradient and no fixed formula. It needs only a way to score each candidate. In addition, GP can discover the *structure* of a solution, not just its numbers. That is its main advantage.

Summary

- A GA evolves fixed-length strings; GP evolves variable-size trees.
- GP is used when the answer is a formula or program, not just a value.
- GP uses the same loop and operators as the GA.
- GP can find the *structure* of a solution, not only its numbers.

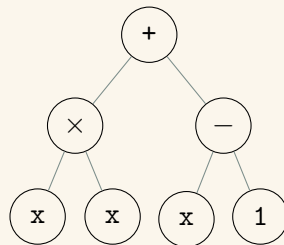
2. Basic Terms

Genetic Programming uses a small vocabulary. Each term has a simple meaning.

- **Tree:** the picture of a program. It is the chromosome in GP. A tree is read from the leaves up to the root.
- **Node:** one point in the tree. Every node is either a function or a terminal.
- **Terminal:** a leaf node. It is an input variable or a constant. Examples: x , 1, 2.
- **Function:** an internal node. It performs an operation on the nodes below it. Examples: $+$, $-$, $\times$.
- **Terminal set:** the list of all terminals allowed in a tree.
- **Function set:** the list of all functions allowed in a tree.
- **Depth:** the number of levels in the tree, from the root to the deepest leaf.

As in the GA, we may speak of a **genotype** and a **phenotype**. The genotype is the tree itself. The phenotype is the formula (and the behaviour) that the tree represents. To read a tree, we start at the leaves and apply each function on the way up to the root.

Example 2.1. The tree below represents the formula $(x \times x) + (x - 1)$. Evaluate it at $x = 3$.



Work from the leaves up, with $x = 3$. The left branch gives $x \times x = 9$. The right branch gives $x - 1 = 2$. The root gives $9 + 2 = 11$.

2.1 Fitness

The **fitness** of a tree measures how well its formula fits the problem. In symbolic regression, we run the tree on each data point and compare its output with the true value. A smaller total error means a better tree. To turn the error into a fitness that we *maximise*, a common choice is

$$F = \frac{1}{1 + E}, \quad E = \sum_i |g(x_i) - y_i|,$$

where g is the tree's formula and E is the total absolute error.

Example 2.2. Our running data comes from the target $t(x) = x^2 + 1$, giving the points below.

x	0	1	2	3
true $y = t(x)$	1	2	5	10

Consider a population of four trees. For each tree we compute the total error E and the fitness $F = 1/(1 + E)$.

Tree	Formula $g(x)$	Total error E	Fitness F
T_1	$x + 1$	8	0.111
T_2	$x \times x$	4	0.200
T_3	x	12	0.077
T_4	$(x + 1) \times x$	4	0.200

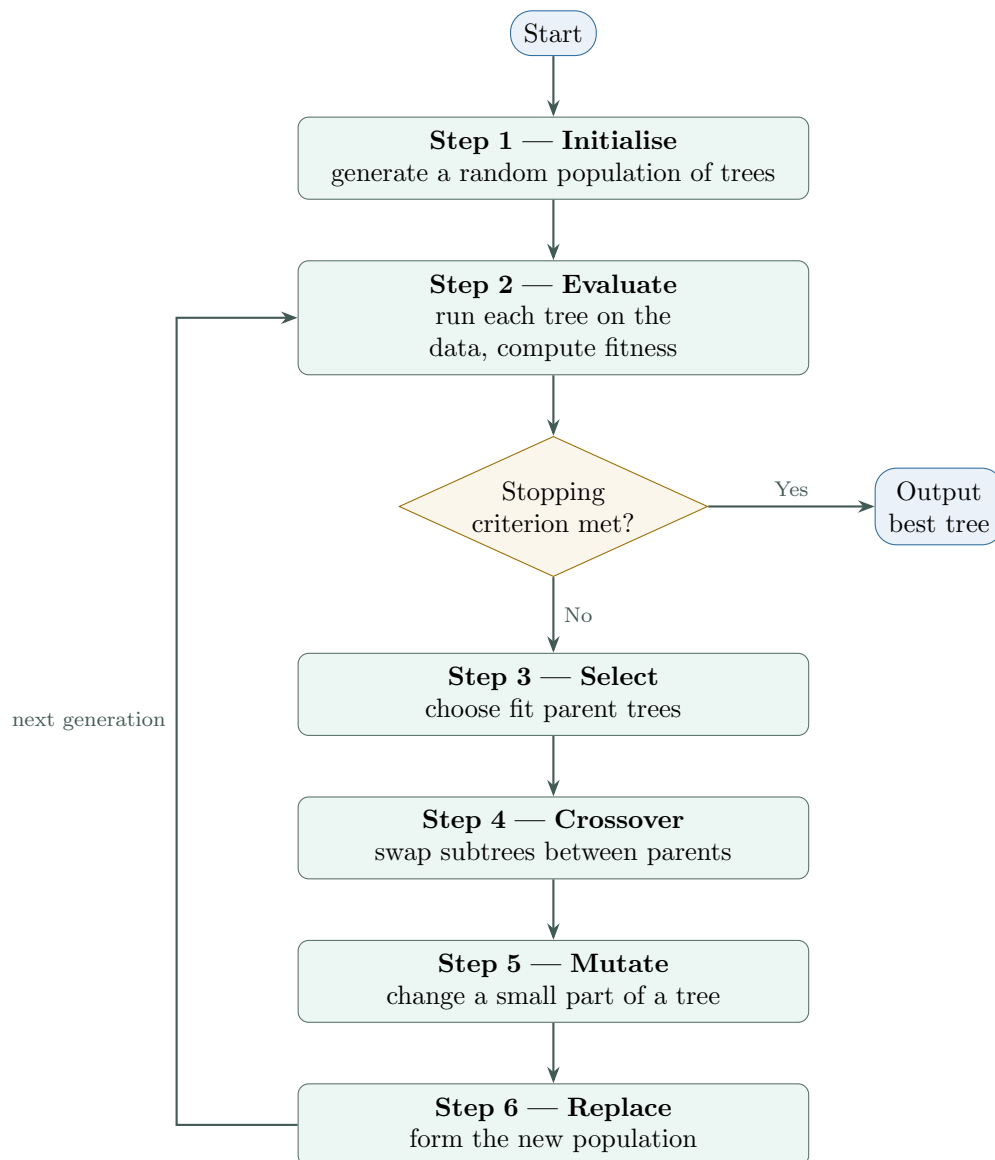
Here T_2 and T_4 are the fittest trees, and T_3 is the weakest. No tree fits the data perfectly yet. We will see later how crossover discovers the exact target.

Summary

- A tree is the GP chromosome; terminals are leaves, functions are inner nodes.
- Genotype = the tree; phenotype = the formula it represents.
- Read a tree from the leaves up to the root.
- Fitness in symbolic regression: small error means high fitness, e.g. $F = 1/(1 + E)$.

3. How GP Works

A GP system repeats the same six steps as a GA. Only the representation and the operators differ, because we now work with trees.



Steps 3, 4, and 5 are the three genetic operators. The loop stops after a fixed number of generations, when the best fitness stops improving, or when a good enough tree is found.

3.1 Tree Representation

Every formula has an order of operations, and a tree stores this order clearly. The function sits above its inputs. The inputs hang below it. Because a tree can have any shape, GP can represent formulas of any size. This is the key difference from the fixed-length string of a GA.

3.2 Terminal and Function Sets

Before we start, we fix the building blocks of our trees. The **terminal set** lists the inputs and constants, for example $\{x, 1\}$. The **function set** lists the operations, for example $\{+, -, \times\}$.

Two simple rules guide the choice of these sets.

- **Closure:** every function must accept any value that a subtree can produce. For example, if division is used, it must be made safe, so that dividing by zero does not crash the program (a common fix is to define $a \div 0 = 1$).
- **Sufficiency:** the two sets must be rich enough to express a good solution. If the target needs multiplication and the function set has only $+$ and $-$, no tree can ever fit it.

3.3 Creating the Initial Population

A random first population should contain trees of many shapes and sizes. Three standard methods are used.

- **Full method:** every branch grows to the same maximum depth. Internal nodes are functions, and all leaves lie at the maximum depth. This produces full, bushy trees.
- **Grow method:** a node may be a function or a terminal (until the maximum depth forces a terminal). This produces trees of varied shapes and sizes.
- **Ramped half-and-half:** a range of depth limits is used. At each depth, half the trees are built by the full method and half by the grow method. This gives the most variety and is the standard choice.

3.4 The Fitness Function

As shown in Chapter 2, the fitness measures how well a tree fits the data. We run the tree on each data point, add the absolute errors to get E , and set $F = 1/(1 + E)$. A perfect tree has $E = 0$ and fitness $F = 1$.

3.5 Selection

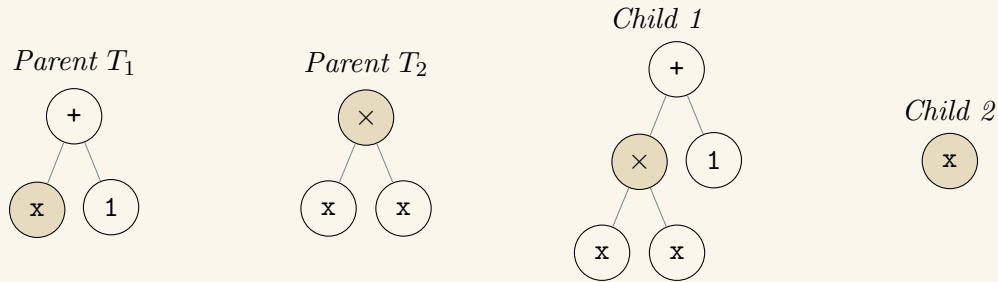
Selection chooses parent trees. Fitter trees should be chosen more often. The methods are the same as in the GA.

- **Roulette-wheel selection:** each tree gets a slice of a wheel proportional to its fitness; a random spin favours larger slices.
- **Tournament selection:** a few trees are drawn at random and the fittest wins. This is simple and popular in GP.
- **Rank selection:** trees are ranked by fitness, and selection uses the rank rather than the raw fitness.

3.6 Crossover (Subtree Swap)

Two parent trees may each contain a good part. Crossover swaps a subtree from each, in the hope that an offspring inherits both good parts. We pick one node in each parent. That node and everything below it form a subtree. We then exchange the two subtrees.

Example 3.1. Take two parents from the running population, $T_1 = x + 1$ and $T_2 = x \times x$. In T_1 we select the left leaf x . In T_2 we select the whole tree $x \times x$. Swapping these two subtrees gives the offspring below.



Child 1 is the tree for $(x \times x) + 1 = x^2 + 1$. Its error is $E = 0$, so its fitness is $F = 1$. Crossover has discovered the exact target function. Child 2 is the single terminal x , a much weaker tree — crossover produces both good and poor offspring, and selection sorts them out later.

3.7 Mutation

Crossover can only reuse subtrees that already exist in the population. Mutation introduces something new. Two common types are used in GP.

- **Subtree mutation:** pick a node, delete its subtree, and grow a new random subtree in its place.
- **Point mutation:** change the label of a single node, for example $+$ becomes $\times$, or the terminal x becomes 1.

Example 3.2. Apply point mutation to the tree for $x + 1$. Change the root $+$ into $\times$. The tree now represents $x \times 1 = x$. A single small change has produced a different program.

3.8 A Complete GP Run

The table below summarises how the running population improved across the operator steps of the first generation. Fitness is $F = 1/(1 + E)$, so the best possible fitness is 1.

Stage	Best error E	Best fitness F
Initial population	4	0.200
After selection	4	0.200
After crossover	0	1.000
After mutation	0	1.000

Selection kept the fitter trees T_2 and T_4 . Crossover then combined T_1 and T_2 to build the tree for $x^2 + 1$, which fits the data exactly. Mutation preserved this best tree while adding variety elsewhere. In a single generation, GP found the target formula.

3.9 Bloat and Its Control

A common problem in GP is **bloat**. Over the generations, trees tend to grow larger and larger, often without any real gain in fitness. Large trees are slow to evaluate and hard to read. Bloat is controlled by setting a maximum tree depth or size, or by adding a small penalty for large trees to the fitness (this penalty is called *parsimony pressure*).

3.10 Parameters and Practical Guidelines

A GP system has a few main parameters.

Parameter	Role	Typical range
N	Population size	100 – 1000
T	Number of generations	50 – 500
p_c	Crossover rate	0.8 – 0.95
p_m	Mutation rate	0.01 – 0.1
max depth	Limit on tree size	6 – 17

GP usually uses larger populations than a GA, because the space of possible trees is very large. Crossover is the main operator, so p_c is high. A maximum depth is set to keep bloat under control.

Summary

- GP uses the same six-step loop as the GA, but on trees.
- Fix the terminal and function sets first; respect closure and sufficiency.
- Build the first population by ramped half-and-half for variety.
- Crossover swaps subtrees; mutation grows a new subtree or changes a node.
- Watch for bloat; limit tree depth or penalise size.

4. Advantages and Limitations

4.1 Advantages

- **Finds structure.** GP discovers the form of a solution, not just its numbers. It can return a human-readable formula.
- **No gradient required.** Like the GA, GP needs only a fitness score.
- **Flexible size.** Trees can grow or shrink, so the size of the solution need not be known in advance.
- **Broad applicability.** The same system fits many tasks — only the terminal set, function set, and fitness change.

4.2 Limitations

- **Bloat.** Trees can grow large without a matching gain in fitness.
- **Computational cost.** GP usually needs large populations and many fitness evaluations, so it is slower than a plain GA.
- **Care with closure.** The function set must be designed so that every tree is valid (for example, safe division).
- **No optimality guarantee.** Like all evolutionary methods, GP returns a good solution, but cannot certify it is the best.

5. Conclusion

Genetic Programming extends the idea of the Genetic Algorithm from evolving values to evolving programs. The candidate solution becomes a tree, built from a terminal set and a function set. The same simple loop — score, select, recombine, perturb, repeat — then searches the space of possible programs.

The power of GP is that it can discover the *structure* of a solution. Given only data, it can grow a formula that fits, as our running example showed when a single crossover produced the exact target $x^2 + 1$. GP is not a universal solver: it can bloat, it needs care in design, and it makes no guarantee of optimality. But as a gradient-free way to search the space of formulas and programs, it is a remarkable and useful tool — and, like the Genetic Algorithm, a fine example of learning from nature.

Python Implementation

A step-by-step Python implementation of Genetic Programming, using the same running example (symbolic regression of $t(x) = x^2 + 1$ with the terminal set $\{x, 1\}$ and the function set $\{+, -, \times\}$), is available as a Google Colab notebook on the author's GitHub.

github.com/kanchan999

Note: After completing these notes, proceed to the companion Question Bank for self-assessment. Best wishes.